

**SMART OS: INTERACTIVE PROCESS SCHEDULING
AND PERFORMANCE ANALYSIS SIMULATOR**

21CSC202J - OPERATING SYSTEMS

A MINI-PROJECT REPORT

Submitted By

WILLIAMS ANTONY A - RA2511026020445

JOSHUA PENIEL D - RA2511026020453

in partial fulfilment for the award of the degree

of

BACHELOR OF TECHNOLOGY

in

**COMPUTER SCIENCE AND ENGINEERING - ARTIFICIAL
INTELLIGENCE AND MACHINE LEARNING**

of

FACULTY OF ENGINEERING AND TECHNOLOGY



SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

RAMAPURAM CAMPUS, CHENNAI-600089

OCTOBER 2026

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

(Deemed to be University U/S 3 of UGC Act, 1956)

BONAFIDE CERTIFICATE

Certified that this mini project report titled **“SMART OS: INTERACTIVE PROCESS SCHEDULING AND PERFORMANCE ANALYSIS SIMULATOR”** is the bonafide work of **“WILLIAMS ANTONY A- RA2511026020445, JOSHUA PENIEL D – RA2511026020453”** carried out at SRM Institute of Science and Technology, Ramapuram Campus, Chennai.

SIGNATURE

Dr. Jananee V. M.E.,PhD.

Assistant Professor,

**Computer Science and
Engineering – AIML & AI**

**SRM Institute of Science and
Technology,**

**Ramapuram Campus,
Chennai.**

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

RAMAPURAM, CHENNAI - 600089

DECLARATION

We hereby declare that the entire work contained in this mini project report titled “**SMART OS: INTERACTIVE PROCESS SCHEDULING AND PERFORMANCE ANALYSIS SIMULATOR**” has been carried out by **WILLIAMS ANTONY A, JOSHUA PENIEL D** at SRM Institute of Science and Technology, Ramapuram Campus, Chennai - 600089.

Place: Chennai

Date:

ABSTRACT

The Smart OS Process Scheduler Simulator is an operating systems mini-project designed to demonstrate how a CPU scheduler selects and executes processes. The system accepts process parameters such as process identifier, arrival time, burst time, and priority, and simulates multiple scheduling policies including First-Come-First-Served (FCFS), Shortest Job First (SJF), Priority Scheduling, and Round Robin. For every selected policy, the simulator generates a textual Gantt chart and calculates important performance measures such as completion time, turnaround time, waiting time, response time, average waiting time, average turnaround time, and context-switch count. The project provides a direct way to compare scheduling behaviour using the same workload and makes core operating-system concepts easier to understand through visible execution traces. The implementation is intentionally lightweight and uses standard C programming constructs, arrays, functions, queues, and time calculations so that it can be compiled and demonstrated in a typical laboratory environment. The design also provides a foundation for future graphical visualization, additional preemptive algorithms, and automated workload generation.

TABLE OF CONTENTS

ABSTRACT	5
CHAPTER 1 - INTRODUCTION	9
1.1 Overview	9
1.2 Problem Statement	9
1.3 Objective of the Project	9
1.4 Scope of the Project	9
CHAPTER 2 - SOFTWARE AND HARDWARE SPECIFICATIONS	10
2.1 Hardware Requirements	10
2.2 Software Requirements	10
2.3 Development Environment	10
CHAPTER 3 - PROJECT DESCRIPTION	11
3.1 Introduction	11
3.2 Architecture Diagram	11
3.3 Process Scheduler Simulation Flow	12
3.4 Algorithms Used	12
3.4.1 FCFS	12
3.4.2 SJF / SJN	13
3.4.3 Priority Scheduling	13
3.4.4 Round Robin	13
3.5 Advantages of the Algorithms Used	13
3.6 Explanation of the Project	13
3.7 Input Parameters	14
3.8 Sample Input	14
3.9 Output Results	14
3.10 Advantages of the Proposed Simulator	15
3.11 Limitations	15
3.12 Future Enhancements	16
3.13 Conclusion	16
APPENDIX - SOURCE CODE	17
REFERENCES	20

LIST OF TABLES

Table 2.1 Hardware Requirements	10
Table 2.2 Software Requirements	10
Table 3.1 Process Input Parameters	14
Table 3.2 Scheduling Algorithms Covered	12
Table 3.3 Sample Performance Results	15

LIST OF FIGURES

Figure 3.1 System Architecture Diagram	11
Figure 3.2 Process Scheduler Simulation Flow	12
Figure 3.3 Sample Round Robin Gantt Chart	14
Figure 3.4 Sample Performance Comparison	15

LIST OF ABBREVIATIONS

Abbreviation	Meaning
OS	Operating System
CPU	Central Processing Unit
FCFS	First-Come-First-Served
SJF	Shortest Job First
SJN	Shortest Job Next
RR	Round Robin
AT	Arrival Time
BT	Burst Time
CT	Completion Time
TAT	Turnaround Time
WT	Waiting Time
RT	Response Time
PCB	Process Control Block

CHAPTER 1

INTRODUCTION

1.1 Overview

Process scheduling is a core function of an operating system. When multiple processes are ready to use the CPU, the scheduler selects one process according to a scheduling policy. The selected process receives CPU service for a period of time, after which another process may be selected. The policy used by the scheduler directly affects waiting time, turnaround time, response time, CPU utilization, fairness, and the number of context switches.

This mini-project develops an interactive simulator that makes these scheduling decisions visible. Instead of executing real operating-system processes, the simulator models the behaviour of a CPU scheduler using process arrival times, burst times, priorities, and a configurable time quantum. The same input workload can be evaluated under different algorithms so that the effect of each policy can be observed and measured.

1.2 Problem Statement

Students often learn scheduling algorithms from formulas and handwritten Gantt charts. While this is useful for examinations, it does not always provide an intuitive understanding of how the ready queue changes over time, how preemption works, or how scheduling choices influence performance metrics. There is therefore a need for a simple simulator that accepts process data, performs scheduling automatically, and reports the resulting execution order and timing metrics in a consistent format.

1.3 Objective of the Project

- Implement a reusable process-scheduling simulation engine in C.
- Simulate FCFS, SJF, Priority Scheduling, and Round Robin algorithms.
- Generate a Gantt chart representation of CPU execution order.
- Calculate CT, TAT, WT, RT, average waiting time, and average turnaround time.
- Count scheduling transitions to illustrate the effect of context switching.
- Provide a common test workload so that algorithm behaviour can be compared fairly.

1.4 Scope of the Project

The scope is limited to CPU scheduling simulation and performance analysis. The project does not modify the host operating system kernel or create real kernel-level processes. The modular design can be extended later with preemptive policies, multilevel queues, graphical charts, CSV export, and workload generation.

CHAPTER 2

SOFTWARE AND HARDWARE SPECIFICATIONS

2.1 Hardware Requirements

Component	Minimum Requirement	Recommended Requirement
Processor	Intel Core i3 / equivalent dual-core	Intel Core i5 / equivalent or above
RAM	4 GB	8 GB or above
Storage	200 MB free space	500 MB free space
Display	1366 x 768	1920 x 1080
Input	Keyboard	Keyboard and mouse

2.2 Software Requirements

Software	Purpose
Operating System	Windows / Linux / macOS for compiling and running the simulator
C Compiler	GCC, MinGW, Clang, or another C99/C11 compatible compiler
IDE / Editor	Visual Studio Code, Code::Blocks, Dev-C++, or equivalent
Terminal	Used to enter process data and view scheduling results

The project uses only standard C programming constructs and therefore does not depend on external libraries. This keeps the implementation portable and suitable for an operating systems laboratory environment. The program can be compiled as a console

application and can later be connected to a graphical front end without changing the core scheduling logic.

2.3 Development Environment

- Programming Language: C
- Compiler Standard: C99/C11
- Development Tool: Visual Studio Code or equivalent IDE
- Version Control: Git/GitHub (optional)
- Operating System for development: Windows 11 / Linux

CHAPTER 3

PROJECT DESCRIPTION

3.1 Introduction

The simulator is organised as a small scheduling framework. The user first enters the number of processes and their attributes. The program then presents a menu from which a scheduling algorithm can be selected. The scheduler applies the selected policy, keeps track of the current CPU time and remaining burst time where required, and records the execution segments. Once all processes are complete, timing metrics are derived from the completion and start times.

The implementation uses separate functions for each scheduling algorithm. This modular structure follows the same principle used in operating-system design: the scheduling policy is isolated from the common process representation and metric calculation. Consequently, new scheduling policies can be added without rewriting the entire program.

3.2 Architecture Diagram

SMART OS - PROCESS SCHEDULER ARCHITECTURE

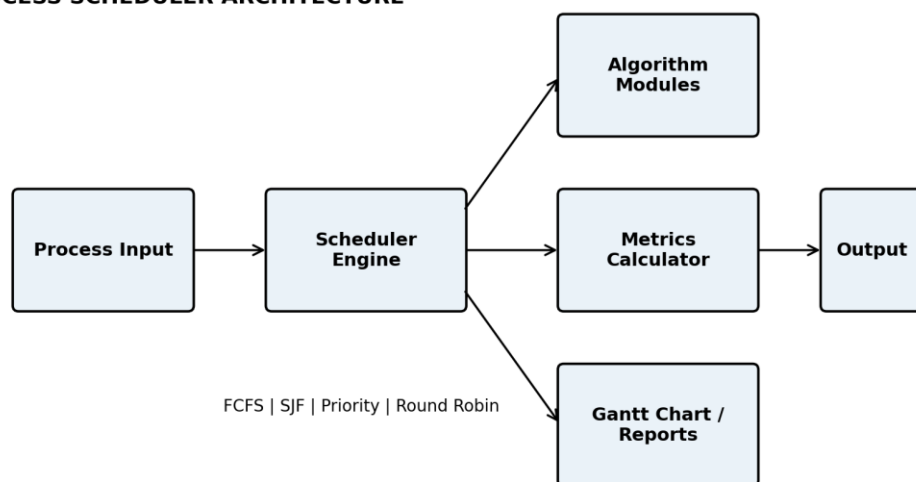


Figure 3.1: System Architecture Diagram

The input layer captures process information. The scheduler engine selects the requested policy and invokes the corresponding algorithm module. The metrics calculator derives completion time, turnaround time, waiting time, and response time. Finally, the Gantt chart and summary report are displayed to the user.

3.3 Process Scheduler Simulation Flow

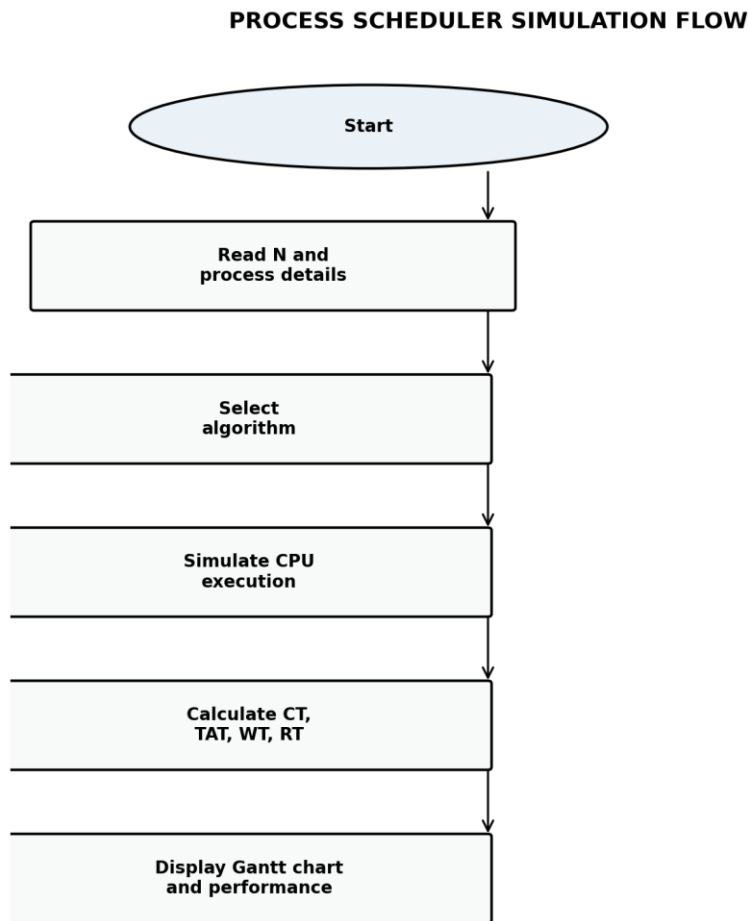


Figure 3.2: Process Scheduler Simulation Flow

3.4 Algorithms Used

Algorithm	Core Idea	Main Characteristic
FCFS	Processes are served in order of	Simple and non-

Algorithm	Core Idea	Main Characteristic
	arrival.	preemptive
SJF / SJN	Among available processes, select the shortest burst time.	Reduces waiting time for many workloads
Priority	Among available processes, select the highest-priority process.	Supports priority-based service
Round Robin	Give each ready process a fixed time quantum in cyclic order.	Preemptive and fairness-oriented

3.4.1 FCFS

First-Come-First-Served schedules the process that arrives first. Once a process starts, it runs until completion. The algorithm is simple because it can be implemented using a queue. Its main limitation is the convoy effect, where short processes may wait behind a long process.

3.4.2 SJF / SJN

Shortest Job First selects the available process with the smallest burst time. In this project it is implemented as a non-preemptive policy. When two processes have the same burst time, the lower process index is used as a deterministic tie-breaker. The policy can reduce average waiting time for many workloads, but long jobs may wait if short jobs continue to arrive.

3.4.3 Priority Scheduling

Priority scheduling selects the ready process with the highest priority. For this project, a smaller numerical priority value denotes higher

priority. The selected process runs to completion in the non-preemptive implementation. Starvation can occur when high-priority processes repeatedly enter the ready queue.

3.4.4 Round Robin

Round Robin is a preemptive scheduling policy. Each ready process receives at most one time quantum before the scheduler moves to the next process. If a process does not finish within the quantum, its remaining burst time is placed back in the ready queue. The algorithm is suitable for interactive systems because no ready process is allowed to occupy the CPU indefinitely.

3.5 Advantages of the Algorithms Used

- FCFS is easy to understand and provides predictable arrival-order execution.
- SJF can produce low average waiting time when burst lengths are known or estimated.
- Priority scheduling allows important or urgent processes to receive earlier service.
- Round Robin provides time sharing and helps maintain responsiveness across multiple ready processes.

3.6 Explanation of the Project

The process record contains process ID, arrival time, burst time, priority, completion time, turnaround time, waiting time, response time, remaining burst time, and the first CPU start time. The common formulae used by the simulator are:

$$\begin{aligned}\text{Turnaround Time (TAT)} &= \text{Completion Time (CT)} \\ &- \text{Arrival Time (AT)} \\ \text{Waiting Time (WT)} &= \text{Turnaround Time (TAT)} -\end{aligned}$$

Burst Time (BT)
 Response Time (RT) = First Start Time -
 Arrival Time (AT)
 Average Waiting Time = Sum of WT / Number of
 Processes
 Average Turnaround Time = Sum of TAT / Number
 of Processes

For preemptive Round Robin, response time is recorded when the process receives the CPU for the first time. Completion time is recorded when the remaining burst becomes zero. For the non-preemptive algorithms, first start time is the time at which the selected process begins its execution.

3.7 Input Parameters

Field	Description	Example
Process ID	Unique label for a process.	P1
Arrival Time	Time at which process enters the ready queue.	2
Burst Time	CPU service time required by the process.	8
Priority	Numerical priority; lower value means higher priority in this project.	1
Time Quantum	Used only for Round Robin.	2

Table 3.1: Process Input Parameters

3.8 Sample Input

Process	Arrival Time	Burst Time	Priority
P1	0	5	2

Process	Arrival Time	Burst Time	Priority
P2	1	3	1
P3	2	8	3
P4	3	2	2

For the Round Robin demonstration, the time quantum is 2 time units. The same workload is evaluated under all four scheduling algorithms so that the differences are attributable to the scheduling policy rather than changes in the input data.

3.9 Output Results

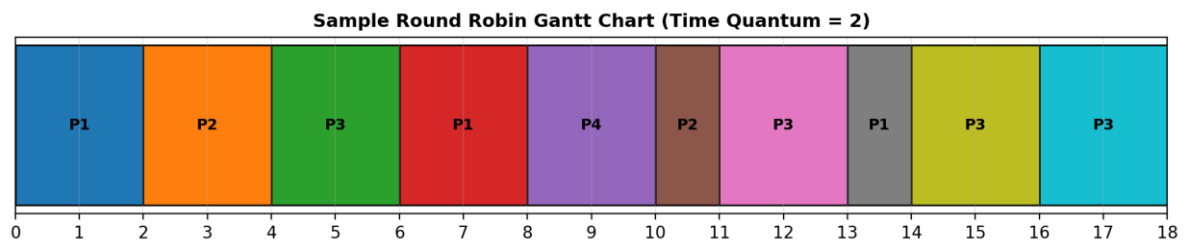


Figure 3.3: Sample Round Robin Gantt Chart

Algorithm	Avg WT	Avg TAT	Avg RT	Context Switches*
FCFS	5.75	10.25	5.75	3
SJF	2.75	7.50	2.75	3
Priority	4.75	9.75	4.75	3
Round Robin (q=2)	7.25	11.75	2.00	9

Table 3.3: Sample Performance Results

*The context-switch count is shown for the simulated execution trace, counting each transition between distinct process execution segments.

A real operating system may include additional context-switch overhead that is not represented by this educational simulator.

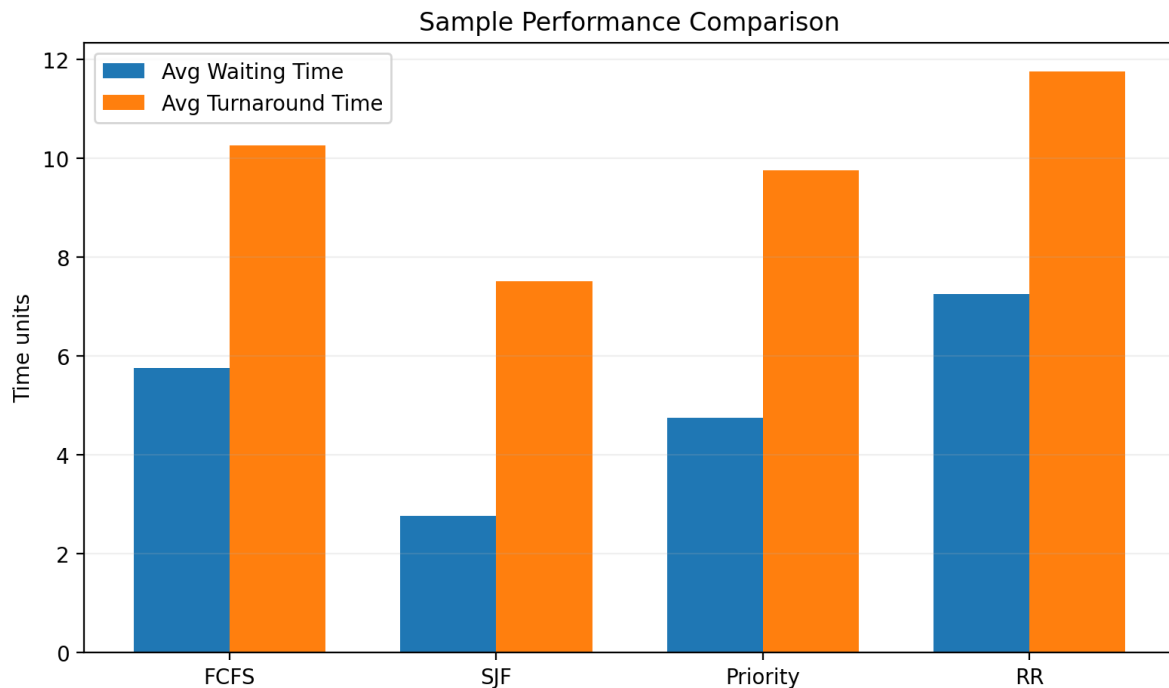


Figure 3.4: Sample Performance Comparison

3.10 Advantages of the Proposed Simulator

- Provides a single interface for multiple scheduling algorithms.
- Makes scheduling decisions visible through a Gantt chart.
- Automatically calculates common scheduling metrics and reduces arithmetic errors.
- Uses the same workload for fair algorithm comparison.
- Has a modular structure that supports future scheduling policies.
- Can be used as a teaching, demonstration, and viva-preparation tool.

3.11 Limitations

- The simulator models scheduling behaviour and does not replace the host operating system scheduler.
- The current implementation uses non-preemptive SJF and non-preemptive priority scheduling.
- Context-switch time is not charged as a separate CPU cost.

- There is no persistent database or graphical desktop interface in the base version.
- Input is assumed to be valid numeric process data.

3.12 Future Enhancements

- Add Shortest Remaining Time First and preemptive Priority Scheduling.
- Add Multilevel Queue and Multilevel Feedback Queue scheduling.
- Build a JavaFX or web dashboard with animated Gantt charts.
- Export reports to CSV or PDF.
- Add random workload generation and batch benchmarking.
- Include configurable context-switch cost and CPU idle-time analysis.

3.13 Conclusion

The Smart OS Process Scheduler Simulator demonstrates the working principles of CPU scheduling through a compact and reproducible implementation. By entering the same processes and changing only the scheduling policy, users can observe differences in execution order and performance measures. The project therefore connects the theoretical study of process scheduling with a practical simulation that is easy to compile, test, extend, and demonstrate in an operating systems laboratory.

APPENDIX

SOURCE CODE

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX 50

typedef struct {
    char id[10];
    int at, bt, pr;
    int rt, rem, ct, tat, wt, first;
} Process;

typedef struct {
    char id[10];
    int start, end;
} Segment;

void reset(Process p[], int n) {
    for (int i = 0; i < n; i++) {
        p[i].rem = p[i].bt;
        p[i].ct = p[i].tat = p[i].wt = 0;
        p[i].rt = -1;
        p[i].first = -1;
    }
}

void sortByArrival(Process p[], int n) {
    for (int i = 0; i < n - 1; i++)
        for (int j = i + 1; j < n; j++)
            if (p[j].at < p[i].at) {
                Process t = p[i]; p[i] = p[j];
                p[j] = t;
            }
}

void addSeg(Segment g[], int *gc, const char
```

```

*id, int s, int e) {
    if (s == e) return;
    if (*gc > 0 && strcmp(g[*gc - 1].id, id) ==
0 && g[*gc - 1].end == s)
        g[*gc - 1].end = e;
    else {
        strcpy(g[*gc].id, id);
        g[*gc].start = s;
        g[*gc].end = e;
        (*gc)++;
    }
}

void finish(Process p[], int n) {
    for (int i = 0; i < n; i++) {
        p[i].tat = p[i].ct - p[i].at;
        p[i].wt = p[i].tat - p[i].bt;
        p[i].rt = p[i].first - p[i].at;
    }
}

void fcfs(Process p[], int n, Segment g[], int
*gc) {
    reset(p, n);
    sortByArrival(p, n);
    int time = 0;
    for (int i = 0; i < n; i++) {
        if (time < p[i].at) time = p[i].at;
        p[i].first = time;
        addSeg(g, gc, p[i].id, time, time +
p[i].bt);
        time += p[i].bt;
        p[i].ct = time;
    }
    finish(p, n);
}

int chooseSJF(Process p[], int n, int time, int
used[]) {

```

```

    int idx = -1;
    for (int i = 0; i < n; i++)
        if (!used[i] && p[i].at <= time)
            if (idx == -1 || p[i].bt <
p[idx].bt ||
                    (p[i].bt == p[idx].bt && p[i].at
< p[idx].at)) idx = i;
    return idx;
}

void sjf(Process p[], int n, Segment g[], int
*gc) {
    reset(p, n);
    int used[MAX] = {0}, done = 0, time = 0;
    while (done < n) {
        int k = chooseSJF(p, n, time, used);
        if (k == -1) {
            int next = 1000000;
            for (int i = 0; i < n; i++) if
(!used[i] && p[i].at < next) next = p[i].at;
            time = next;
            continue;
        }
        used[k] = 1; p[k].first = time;
        addSeg(g, gc, p[k].id, time, time +
p[k].bt);
        time += p[k].bt; p[k].ct = time;
        done++;
    }
    finish(p, n);
}

int choosePriority(Process p[], int n, int
time, int used[]) {
    int idx = -1;
    for (int i = 0; i < n; i++)
        if (!used[i] && p[i].at <= time)
            if (idx == -1 || p[i].pr <
p[idx].pr ||

```

```

        (p[i].pr == p[idx].pr && p[i].at
< p[idx].at)) idx = i;
        return idx;
}

void priority(Process p[], int n, Segment g[],
int *gc) {
    reset(p, n);
    int used[MAX] = {0}, done = 0, time = 0;
    while (done < n) {
        int k = choosePriority(p, n, time,
used);
        if (k == -1) {
            int next = 1000000;
            for (int i = 0; i < n; i++) if
(!used[i] && p[i].at < next) next = p[i].at;
            time = next;
            continue;
        }
        used[k] = 1; p[k].first = time;
        addSeg(g, gc, p[k].id, time, time +
p[k].bt);
        time += p[k].bt; p[k].ct = time;
        done++;
    }
    finish(p, n);
}

void roundRobin(Process p[], int n, int q,
Segment g[], int *gc) {
    reset(p, n);
    Process a[MAX];
    memcpy(a, p, sizeof(Process) * n);
    sortByArrival(a, n);
    int qv[MAX * 4], front = 0, rear = 0, time
= 0, next = 0, done = 0;
    int map[MAX];
    for (int i = 0; i < n; i++) map[i] = i;
    while (done < n) {

```

```

        if (front == rear && next < n && time <
a[next].at) time = a[next].at;
        while (next < n && a[next].at <= time)
qv[rear++] = next++;
        if (front == rear) continue;
        int ai = qv[front++];
        int k = map[ai];
        if (p[k].first == -1) p[k].first =
time;

        int run = p[k].rem < q ? p[k].rem : q;
        addSeg(g, gc, p[k].id, time, time +
run);

        time += run; p[k].rem -= run;
        while (next < n && a[next].at <= time)
qv[rear++] = next++;
        if (p[k].rem > 0) qv[rear++] = ai;
        else { p[k].ct = time; done++; }
    }
    finish(p, n);
}

void printResult(Process p[], int n, Segment
g[], int gc) {
    double aw=0, at=0, ar=0;
    printf("\nGANTT CHART\n");
    for (int i = 0; i < gc; i++) printf("| %s
", g[i].id);
    printf("| \n%d", g[0].start);
    for (int i = 0; i < gc; i++) printf("\t%d",
g[i].end);

printf("\n\nProcess\tAT\tBT\tPR\tCT\tTAT\tWT\tRT\n");
    for (int i=0;i<n;i++) {

printf("%s\t%d\t%d\t%d\t%d\t%d\t%d\t%d\n",

p[i].id,p[i].at,p[i].bt,p[i].pr,p[i].ct,p[i].ta
t,p[i].wt,p[i].rt);

```

```

        aw += p[i].wt; at += p[i].tat; ar +=
p[i].rt;
    }
    printf("\nAverage Waiting Time      :
%.2f\n", aw/n);
    printf("Average Turnaround Time : %.2f\n",
at/n);
    printf("Average Response Time      : %.2f\n",
ar/n);
    printf("Context Switches           : %d\n", gc
> 0 ? gc - 1 : 0);
}

int main() {
    Process p[MAX];
    Segment g[MAX * 4];
    int n, choice, q, gc;
    printf("SMART OS - PROCESS SCHEDULER
SIMULATOR\n");
    printf("Enter number of processes: ");
    scanf("%d", &n);
    if (n < 1 || n > MAX) return 1;

    for (int i = 0; i < n; i++) {
        sprintf(p[i].id, "P%d", i + 1);
        printf("\nProcess %s\n", p[i].id);
        printf("Arrival Time : "); scanf("%d",
&p[i].at);
        printf("Burst Time      : "); scanf("%d",
&p[i].bt);
        printf("Priority         : "); scanf("%d",
&p[i].pr);
    }

    printf("\n1. FCFS\n2. SJF\n3. Priority\n4.
Round Robin\n");
    printf("Choose algorithm: "); scanf("%d",
&choice);
    gc = 0;

```

```
    if (choice == 1) fcfs(p,n,g,&gc);
    else if (choice == 2) sjf(p,n,g,&gc);
    else if (choice == 3) priority(p,n,g,&gc);
    else if (choice == 4) {
        printf("Time Quantum: "); scanf("%d",
&q);
        if (q <= 0) return 1;
        roundRobin(p,n,q,g,&gc);
    } else {
        printf("Invalid choice.\n"); return 1;
    }
    printResult(p,n,g,gc);
    return 0;
}
```

REFERENCES

1. [1] Abraham Silberschatz, Peter B. Galvin and Greg Gagne, Operating System Concepts, Wiley.
2. [2] Andrew S. Tanenbaum and Herbert Bos, Modern Operating Systems, Pearson.
3. [3] William Stallings, Operating Systems: Internals and Design Principles, Pearson.
4. [4] Course notes and laboratory exercises for Operating Systems - Process Scheduling.
5. [5] ISO/IEC 9899:2018, Programming Languages - C.